



KELOMPOK 8



ITS
SEMANGAT
BARU

myITS
learning

Software Supply Chain Security

(SBOM Generation + Artifact Signing + Vulnerability Scanning)

Anggota



Dian Anggraeni P

5027231016



Acintya S

5027231020



Tsaldia Hukma C

5027231036



Callista Meyra A

5027231060

The Future of DevSecOps: Supply Chain Security



Masalah saat ini:

Pipeline biasa (Code → Build → Deploy) buta terhadap dependensi di dalam aplikasi.



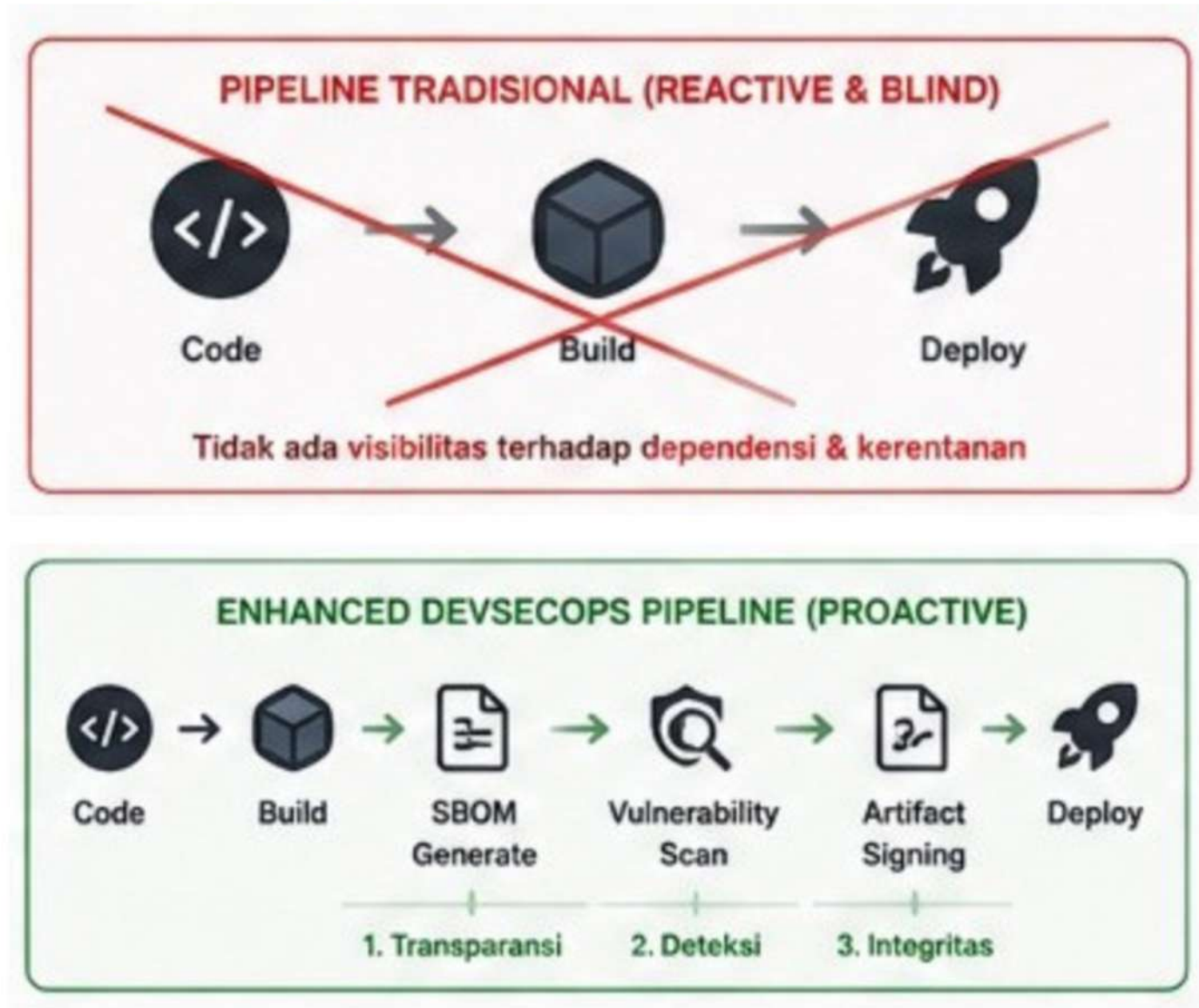
Risiko Nyata:

Serangan rantai pasok (Supply Chain Attacks) seperti Log4Shell & SolarWinds.



Solusi (Berdasarkan Riset):

1. Transparansi: Generate SBOM otomatis (Software Bill of Materials).
2. Deteksi: Scanning kerentanan berbasis SBOM.
3. Integritas: Verifikasi Artifact Signing.



Dari Blind Pipeline ke Transparent & Secure Pipeline

Aplikasi Node.js dengan "Intentional Vulnerabilities"

Aplikasi:

Task Management REST API (Express.js).

Skenario:

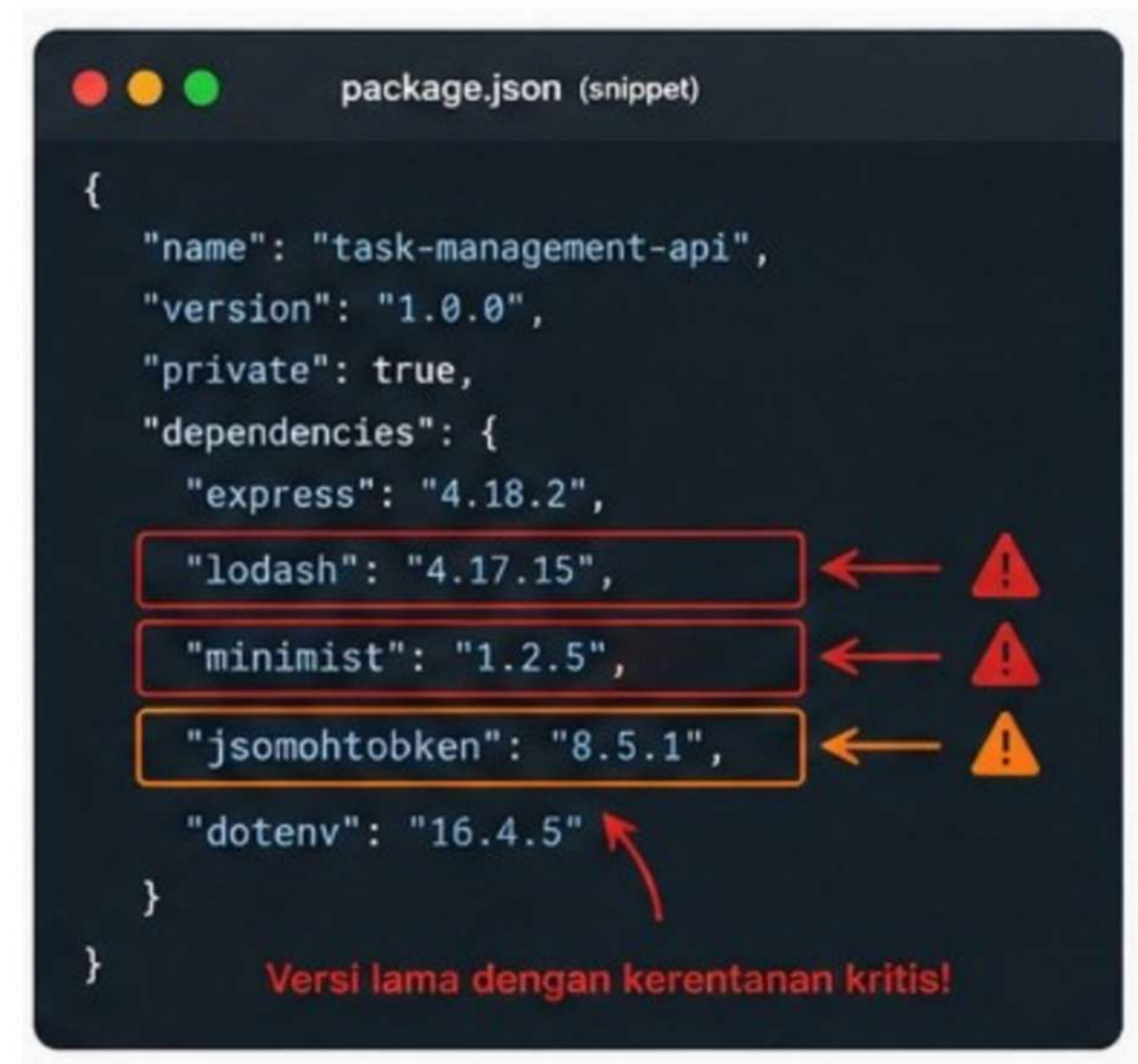
Menguji kepekaan pipeline CI/CD terhadap celah keamanan kritis.

Vulnerable Dependencies:

- lodash v4.17.15 → Command Injection (CRITICAL)
- minimist v1.2.5 → Prototype Pollution (CRITICAL)
- jsonwebtoken v8.5.1 → Remote Code Execution (HIGH)

Hipotesis:

Pipeline tradisional akan meloloskan aplikasi ini ke Production, sementara Enhanced Pipeline kami harus mampu memblokirnya.



```
package.json (snippet)

{
  "name": "task-management-api",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "express": "4.18.2",
    "lodash": "4.17.15",
    "minimist": "1.2.5",
    "jsonwebtoken": "8.5.1",
    "dotenv": "16.4.5"
  }
}
```

Annotations in the image:

- Red boxes around "lodash": "4.17.15", "minimist": "1.2.5", and "jsonwebtoken": "8.5.1".
- Red arrows pointing to each of these three dependencies.
- Red warning triangle icons to the right of each of these three dependencies.
- A red arrow pointing to "dotenv": "16.4.5".
- Red text at the bottom: "Versi lama dengan kerentanan kritis!"

Secure Containerization



Multi-Stage Build:

Memisahkan tahap build dan produksi untuk memperkecil ukuran image dan membuang tools yang tidak perlu.

Production-Only Dependencies:

Menggunakan `npm ci --omit=dev` agar dependensi testing/development tidak ikut masuk ke production.

Non-Root User:

Aplikasi dijalankan sebagai appuser (bukan admin/root) → Meminimalisir dampak jika terjadi eksekusi kode jarak jauh (RCE).

Non-Root User:

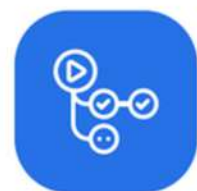
Aplikasi dijalankan sebagai appuser (bukan admin/root) → Meminimalisir dampak jika terjadi eksekusi kode jarak jauh (RCE).

Implementasi SBOM pada CI/CD Pipeline



Syft (Software Bill of Materials Generator)

Syft merupakan open-source CLI tool yang digunakan untuk menghasilkan SBOM dari Docker Image dalam format CycloneDX dan SPDX. Syft dapat diinstal secara langsung atau dijalankan melalui GitHub Actions



GitHub Actions

GitHub Actions

GitHub Actions merupakan platform CI/CD yang mengotomatisasi proses generate, verifikasi, validasi, dan upload SBOM melalui workflow berbasis YAML



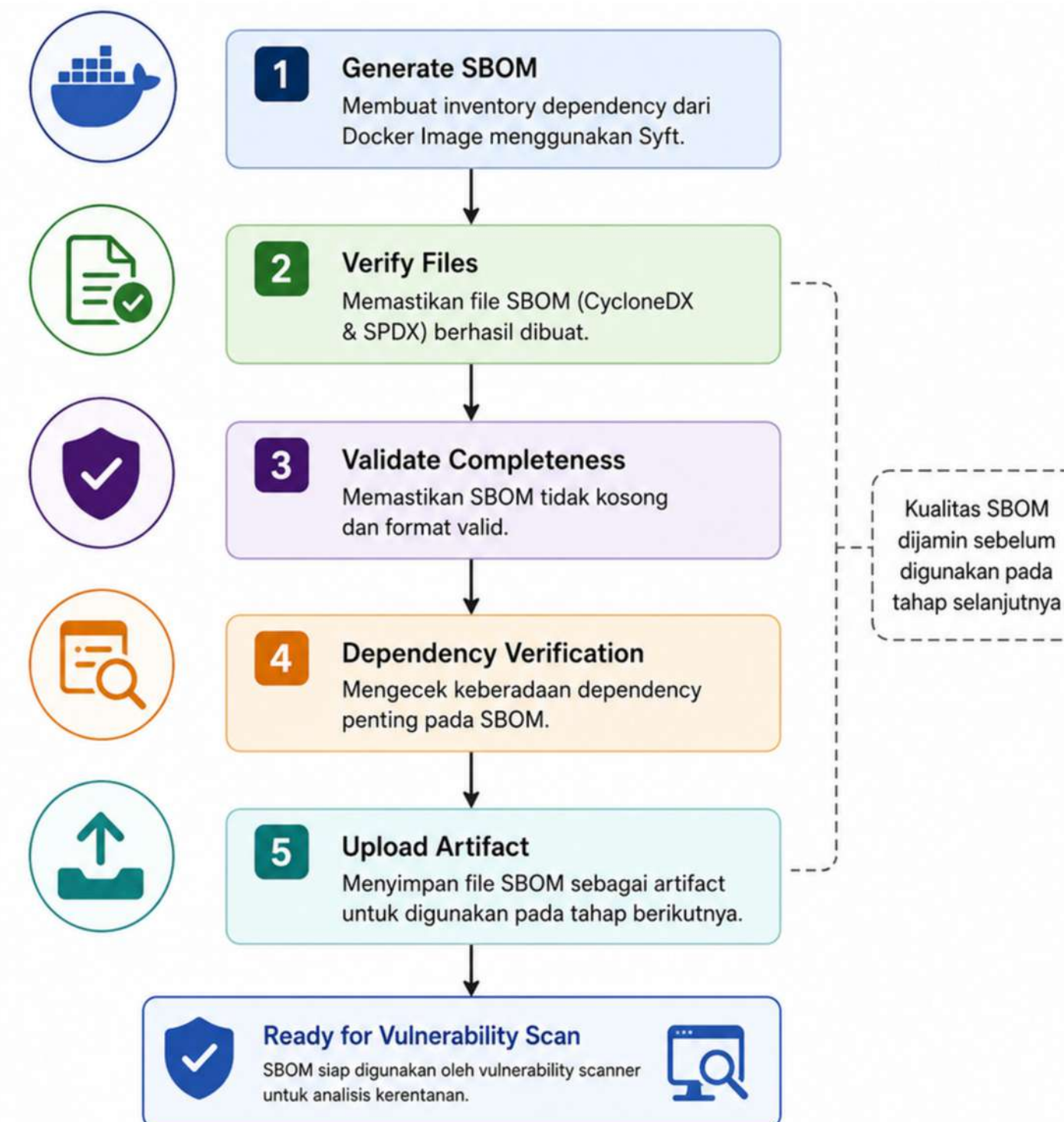
CycloneDX

CycloneDX merupakan standar format SBOM yang dikembangkan oleh OWASP untuk mendokumentasikan komponen perangkat lunak dan mendukung keamanan software supply chain



SPDX (Software Package Data Exchange)

SPDX merupakan standar format SBOM yang dikembangkan oleh Linux Foundation untuk mendokumentasikan paket perangkat lunak, lisensi, dan dependency agar kompatibel dengan berbagai tools keamanan



Hasil Implementasi SBOM

Generate SBOM summary

✓ **SBOM Verification Results**

Verification	Result
SBOM Files Generated	✓
CycloneDX Components	684
SPDX Packages	309
Dependency Verification	✓
Completeness Validation	✓

📦 **Verified Application Dependencies**

- lodash
- axios
- jsonwebtoken
- minimist
- node-fetch

📊 **Dependency Coverage Metrics**

Metric	Value
CycloneDX Components	684
SPDX Packages	309
Verified Critical Dependencies	5

📖 **Research Alignment**

This implementation follows the supply chain security recommendations discussed in the selected research papers.

Research Paper	Implementation
Xia et al. (2023)	Automated SBOM generation in CI/CD
O'Donoghue et al. (2024)	Syft + CycloneDX and SPDX comparison

✓ Result: SBOM generation, validation, and reporting completed successfully and provide trusted input for downstream vulnerability scanning activities.

Memastikan proses generate dan validasi SBOM berhasil

Menampilkan dependency penting yang berhasil diidentifikasi

Menampilkan jumlah komponen dan package yang berhasil dianalisis

Membuktikan implementasi sesuai rekomendasi penelitian

Hasil Implementasi SBOM

Coverage Summary

- Application dependencies successfully detected.
- OS-level packages included in SBOM inventory.
- Generated SBOM ready for downstream vulnerability scanning.

SBOM Generation Results

CycloneDX SBOM

- Total components detected: 684
- Format: CycloneDX JSON
- Tool: Syft (Anchore)

SPDX SBOM

- Total packages detected: 309
- Format: SPDX JSON

Format Comparison (O'Donoghue et al., 2024)

Format	Components Detected
CycloneDX	684
SPDX	309

Observation: CycloneDX detected more software components than SPDX in this image.

Job summary generated at run-time

Merangkum cakupan hasil SBOM

Menampilkan hasil pembentukan SBOM

Membandingkan hasil dua format SBOM

Memberikan interpretasi hasil perbandingan

Artifacts			
Produced during runtime			
Name	Size	Digest	
 sbom-cyclonedx	69.4 KB	sha256:a622dbe2e27f1c3683e09c43e7027e4896e597f4d78e4d089f12c14394310d9a	  
 sbom-reports	155 KB	sha256:152c4d09ff4fe43b81bf9d27649165577780690ea69268fdf0bfffadde790f792	  
 sbom-spdx	85.7 KB	sha256:495eff6cd72a223636fa8701c888106e20b03e5146a3073aba4bf3733757544f	  

Integrasi SBOM & Vulnerability Scanning



Separation of Concerns

Pemisahan tool SBOM generation (Syft) dan vulnerability scanning (Trivy) dibanding menggunakan pendekatan all-in-one.

Justifikasi Paper Acuan:

Impacts of Software Bill of Materials (SBOM) Generation on Vulnerability Detection

Eric O'Donoghue
Montana State University
Bozeman, MT, USA
ericodonoghue@montana.edu

Brittany Boley
Montana State University
Bozeman, MT, USA
brittanyboley@montana.edu

Clemente Izurieta
Montana State University
Bozeman, MT, USA
Pacific Northwest National Laboratory
Richland, WA, USA
clemente.izurieta@montana.edu

Ann Marie Reinhold
Montana State University
Bozeman, MT, USA
Pacific Northwest National Laboratory
Richland, WA, USA
annmarie.reinhold@montana.edu

Abstract
The software supply chain (SC) continues to face cybersecurity threats. To assist in securing SCs, Software Bill of Materials (SBOM) has emerged as a pivotal technology. Despite the increasing use of SBOMs, the influence of SBOM generation on vulnerability detection was unaddressed. We created four corpora of SBOMs from 2,513 Docker images by varying SBOM generation tool (Syft, Trivy) and SBOM format (CycloneDX, SPDX). Using these corpora, SBOM analysis tools (Trivy, Grype, CVEs, etc.) were investigated how the reported vulnerabilities for the same software artifact varied when we changed only the SBOM generation tool and format. With the complex nature of SBOM generation and analysis, we expected some variation in reported vulnerabilities. However, we found high variability in vulnerability reporting attributed to SBOM generation. The variation in the quantity of vulnerabilities discovered in the same software artifact highlights the need for rigorous validation and enhancement of SBOM technologies to best secure SCs.

CCS Concepts
• Security and privacy → Software security engineering.

Keywords
Software Supply Chain Security; Software Bill of Materials (SBOM); Vulnerability Detection; Misinformation

ACM Reference Format:
Eric O'Donoghue, Brittany Boley, Clemente Izurieta, and Ann Marie Reinhold. 2024. Impacts of Software Bill of Materials (SBOM) Generation on Vulnerability Detection. In *Proceedings of the 2024 ACM Conference on Software Supply Chain (OSS-SC) Research and Practice (OSS-SC '24)*, June 14–15, 2024, Salt Lake City, UT, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3658888>

This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2024 by the author(s). All rights reserved. This article is intended solely for the personal use of the individual user and is not to be disseminated broadly.
© 2024 Copyright held by the owner(s).
0730-0015/24/06-1000-10\$15.00
<https://doi.org/10.1145/3658888>

O'Donoghue et al. (2024) menunjukkan bahwa:

1. Pilihan tool SBOM secara signifikan mempengaruhi hasil deteksi celah keamanan.
2. Pemisahan proses generation dan scanning memberikan fleksibilitas dan cakupan (coverage) deteksi yang lebih baik.
3. CycloneDX menghasilkan deteksi vulnerability yang lebih konsisten dibanding SPDX.

Konsistensi Deteksi

Riset membuktikan pemisahan tool meminimalisasi blind spot keamanan dan meningkatkan akurasi deteksi.

Format CycloneDX JSON

Menggunakan format CycloneDX JSON sebagai input pemindaian karena tingkat kompatibilitas dan parsing yang lebih cepat oleh scanner.

Vuln Scan & Security Gate



DUAL SCAN MODE

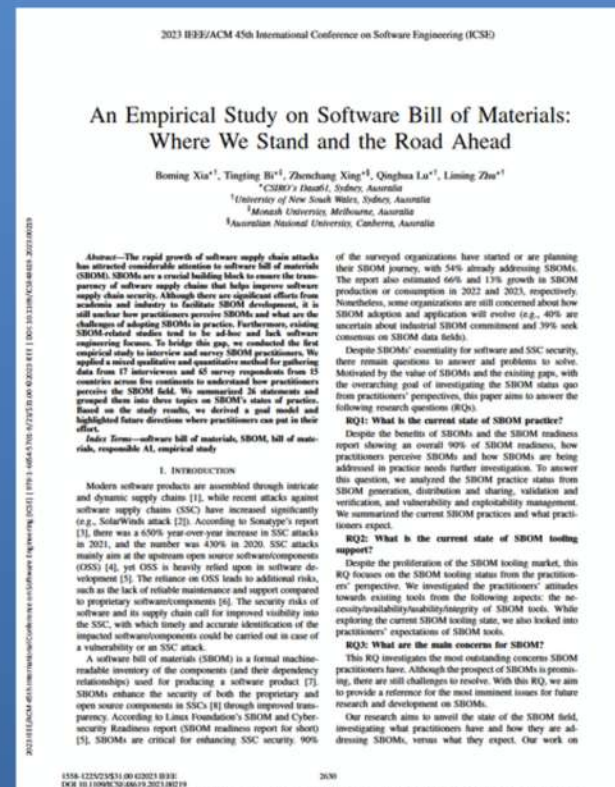
Pipeline menjalankan dua jenis pemindaian sekaligus:

1. Image Scan: untuk mendeteksi paket OS container.
2. SBOM Scan (CycloneDX) untuk mendeteksi library dependensi aplikasi.

SECURITY GATE

1. Critical Severity: Otomatis memblokir pipeline (fail fast)
2. High Severity: Lolos dengan warning pada Github Summary
3. Medium/Low: Dicatat pada log audit

Severity (Keparahan)	Aksi Pipeline
CRITICAL	Block ❌
HIGH	Warning ⚠️
MEDIUM/LOW	Log ⓘ



Justifikasi Paper Acuan:

Xia et al. (2023) menyimpulkan bahwa:

1. Nilai dari sebuah SBOM terletak pada tingkat Actionability (keaktifan tindakan), bukan sekadar compliance pasif.
2. Kebijakan Fail-Fast memastikan SBOM digunakan secara aktif untuk melindungi production environment dari ancaman nyata secara otomatis.

Pengujian Security Gate



enhanced-pipeline.yml

on: push



Vulnerability Scan Results

Komponen: Vulnerability Scanning & Security Gate

Image Scan (Traditional)

Severity	Count
CRITICAL	3
HIGH	48
MEDIUM	35
Total	86

Detailed Findings (CRITICAL & HIGH)

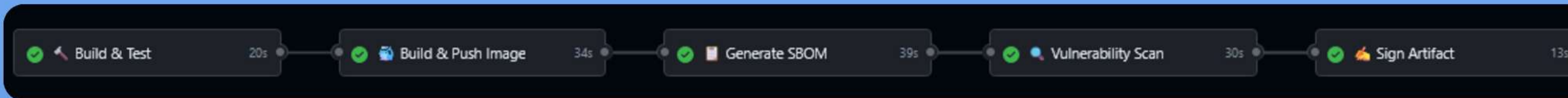
CVE ID	Package	Installed	Fixed	Severity
CVE-2026-31789	libcrypto3	3.3.3-r0	3.3.7-r0	CRITICAL
CVE-2026-31789	libssl3	3.3.3-r0	3.3.7-r0	CRITICAL
CVE-2021-44906	minimist	1.2.5	1.2.6, 0.2.4	CRITICAL
CVE-2025-15467	libcrypto3	3.3.3-r0	3.3.6-r0	HIGH
CVE-2025-69421	libcrypto3	3.3.3-r0	3.3.6-r0	HIGH
CVE-2026-28387	libcrypto3	3.3.3-r0	3.3.7-r0	HIGH
CVE-2026-28388	libcrypto3	3.3.3-r0	3.3.7-r0	HIGH
CVE-2026-28389	libcrypto3	3.3.3-r0	3.3.7-r0	HIGH
CVE-2026-28390	libcrypto3	3.3.3-r0	3.3.7-r0	HIGH
CVE-2025-15467	libssl3	3.3.3-r0	3.3.6-r0	HIGH
CVE-2025-69421	libssl3	3.3.3-r0	3.3.6-r0	HIGH
CVE-2026-28387	libssl3	3.3.3-r0	3.3.7-r0	HIGH
CVE-2026-28388	libssl3	3.3.3-r0	3.3.7-r0	HIGH
CVE-2026-28389	libssl3	3.3.3-r0	3.3.7-r0	HIGH

Dependensi sengaja menggunakan versi outdated dengan CVE yang diketahui di **package.json**.

Hasil Pemindaian: Menemukan 3 CRITICAL Vulnerabilities

Status pipeline FAILED dan deployment diblokir demi mencegah artifact yang rentan masuk ke production.

Remediasi



```
app/Dockerfile
@@ -27,6 +27,7 @@ RUN npm test
27 27 # Stage 2: Production
28 28 # =====
29 29 FROM node:18-alpine AS production
30 + RUN apk update && apk upgrade --no-cache
30 31
31 32 # Metadata labels sesuai OCI Image Spec
32 33 # Label ini akan muncul di SBOM dan membantu traceability
```

REMEDIASI OS CONTAINER

Menambahkan baris **RUN apk update && apk upgrade --no-cache** pada stage produksi di Dockerfile untuk memperbarui OS library.

REMEDIASI NODE.JS

Upgrade minimist dari 1.2.5 ke 1.2.8 (dan dependensi lain seperti lodash, jsonwebtoken, axios, dan node-fetch).

```
28 - "jsonwebtoken": "8.5.1",
29 - "lodash": "4.17.15",
30 - "axios": "0.21.1",
31 - "minimist": "1.2.5",
32 - "node-fetch": "2.6.0"

28 + "jsonwebtoken": "9.0.2",
29 + "lodash": "4.17.21",
30 + "axios": "1.7.9",
31 + "minimist": "1.2.8",
32 + "node-fetch": "2.7.0"
```


Implementasi Artifact Signing & Justifikasi Riset



Gap Keamanan (Problem)

- Pipeline tradisional memiliki risiko *Tampering* (image diubah oleh pihak ketiga di *Registry*).
- Hambatan utama: **Key Management Complexity**. Mengelola kunci privat (**.key**) sangat berisiko jika hilang atau dicuri.



Landasan Paper (Riset)

- **Kalu et al. (2025)**: "An Industry Interview Study of Software Signing..."
- **Temuan Utama**: *Key management* adalah *barrier* nomor satu adopsi penandatanganan. Paper ini merekomendasikan **Keyless Signing**.



Solusi & Implementasi:

- Menggunakan **Cosign (Sigstore)** dengan metode **Keyless Signing**.
- **OIDC Identity**: Menggunakan identitas GitHub Actions untuk menandatangani image tanpa file kunci permanen.
- **Transparency Log**: Bukti penandatanganan dicatat secara permanen di **Rekor**, memungkinkan audit publik yang tidak bisa dimanipulasi.

Evaluasi & Bukti Verifikasi

Perbandingan Metrik (Before vs After)

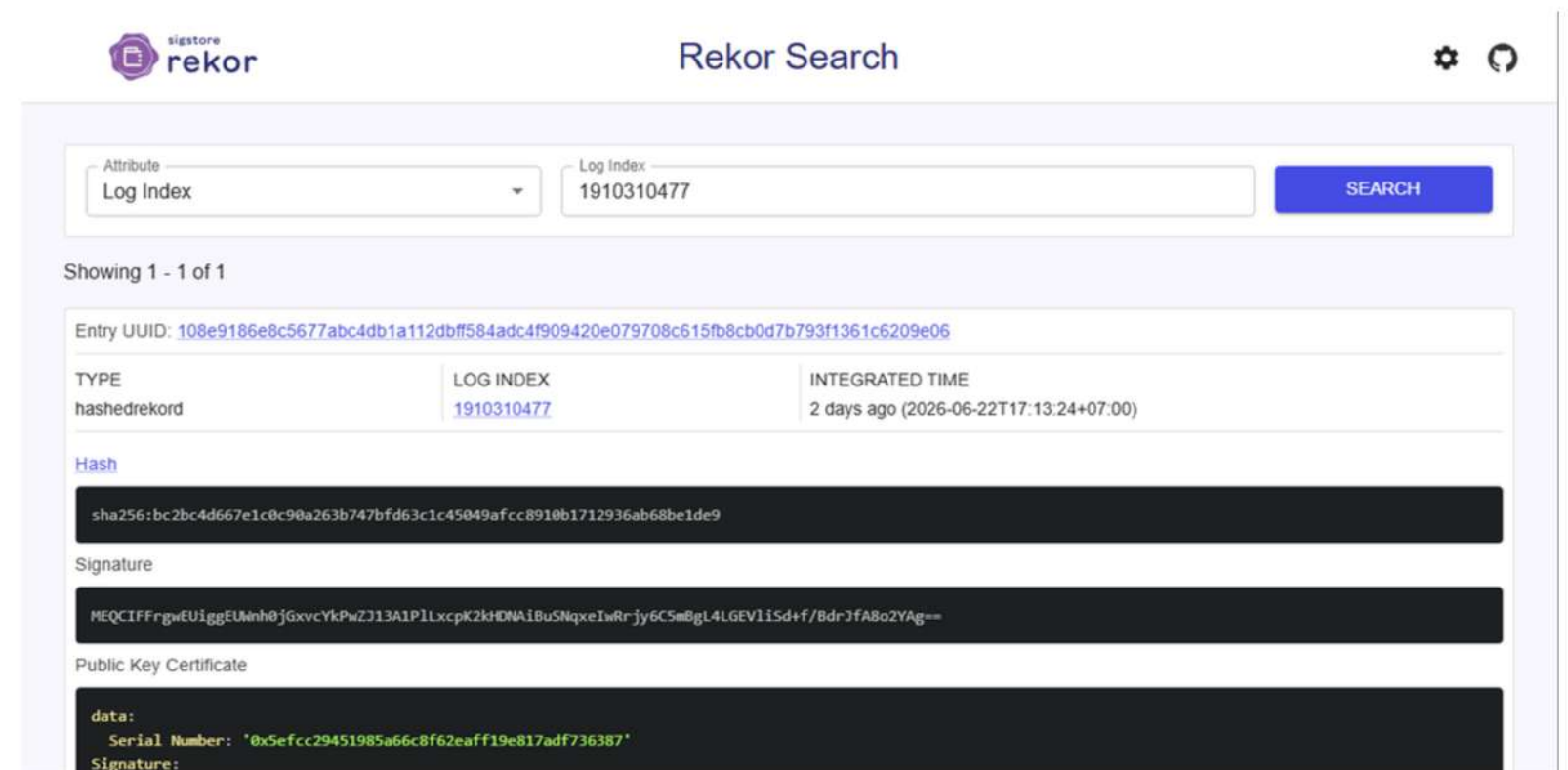
Aspek	Sebelum (Baseline)	Sesudah (Enhanced)
Status Signing	✗ 0% (Unsigned)	✓ 100% (Signed)
Verifikasi	✗ Tidak bisa dibuktikan	✓ Terverifikasi Kriptografis
Traceability	Rendah (Hanya Tag)	Tinggi (Rekor Log Index)

• Live Verification Command:

```
cosign verify --certificate-identity-regexp=".*kelompok-8.*" \
--certificate-oidc-issuer="https://token.actions.githubusercontent.com"
[IMAGE_URL]
```

Bukti Hasil (Evidence):

- **Log Index: #1910310477** (Tercatat di Sigstore Rekord).
- **Sertifikat OIDC:** Terikat pada workflow enhanced-pipeline.yml di repository kelompok.



The screenshot shows the 'Rekor Search' interface. At the top, there's a search bar with 'Log Index' selected as the attribute and '1910310477' entered. A 'SEARCH' button is to the right. Below the search bar, it says 'Showing 1 - 1 of 1'. The results show an entry with the following details:

- Entry UUID: 108e9186e8c5677abc4db1a112dbff584adc4f909420e079708c615fb8cb0d7b793f1361c6209e06
- TYPE: hashedrekord
- LOG INDEX: 1910310477
- INTEGRATED TIME: 2 days ago (2026-06-22T17:13:24+07:00)
- Hash: sha256:bc2bc4d667e1c0c90a263b747bfd63c1c45049afcc8910b1712936ab68be1de9
- Signature: MEQCIFFrgwEuiiggEUMnh0jGxvcYkPwZ313A1P1LxcpK2k4DWAiBuSNqxeIwRrjy6C5mBgl4LGEV1i5d+f/BdrJfA8o2YAg==
- Public Key Certificate: data: Serial Number: '0x5efcc29451985a66c8f62eaff19e817adf736387' Signature:

Kesimpulan:

- Dengan Artifact Signing, kita menjamin **Provenance** (asal-usul) dan **Integrity** (keutuhan). Jika image dimodifikasi di GHCR, proses verifikasi akan gagal secara otomatis.



KELOMPOK 8



ITS
SEMANGAT
BARU

myITS
learning

Thank You

"Security is not a product, but a process."

— Bruce Schneier